



DETAILED_IMPLEMENTATION_TRACKER

HELIXCODE DETAILED IMPLEMENTATION TRACKER

Status: CRITICAL FIXES REQUIRED

Phase: 0 - Build System Restoration

Current Focus: Compilation Error Resolution

Next Milestone: All Tests Passing

DAILY TASK BREAKDOWN - PHASE 0 (Week 1)

DAY 1: CRITICAL COMPILATION ANALYSIS

Date: Day 1 of implementation

Objective: Identify and catalog all compilation errors

Morning Session (4 hours)

Task 1.1: Complete build error catalog

09:00-10:30 - Analyze helix_code/ build errors

```
cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode  
go build -v ./... 2>&1 | tee build_errors_day1.log
```

10:30-12:00 - Categorize errors by severity

Critical (blocking): X11 dependencies, missing packages

High: Mock compilation errors

Medium: Test compilation errors

Low: Warning and deprecation notices

Afternoon Session (4 hours)

Task 1.2: Fix X11 GUI dependencies

13:00-15:00 - Install missing system dependencies

```
sudo apt-get update
```

```
sudo apt-get install -y libx11-dev libxcursor-dev libxrandr-dev  
libxinerama-dev libxi-dev libgl1-mesa-dev
```

15:00-17:00 - Test GUI dependencies

```
go build -v ./applications/desktop/...
go build -v ./applications/terminal_ui/...
```

Deliverables Day 1:

- ☐ Complete build error catalog
 - ☐ X11 dependencies resolved
 - ☐ GUI applications building
 - ☐ Build errors log file
-

DAY 2: MOCK AND TEST FIXES

Date: Day 2 of implementation

Objective: Fix memory mocks and API key test compilation

Morning Session (4 hours)

```
# Task 2.1: Fix memory_mock.go compilation
cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
```

```
09:00-10:30 - Analyze memory_mock.go errors
vim internal/mocks/memory_mock.go
# Fix lines: 688 (ProviderTypeChroma), 1003 (MemoryData), 1037
  (ConversationMessage)
```

```
10:30-12:00 - Define missing types and constants
# Add to internal/providers/types.go if needed
# Add to internal/memory/types.go if needed
# Ensure all imports are correct
```

Afternoon Session (4 hours)

```
# Task 2.2: Fix API key integration test
cd /media/milosvasic/DATA4TB/Projects/HelixCode
```

```
13:00-14:30 - Analyze api_key_integration_test.go
vim isolated_files/api_key_integration_test.go
# Fix line 176: config.NewAPIKeyManager function
# Fix lines 262-293: config.Strategy* constants
```

```
# Fix line 303: helixConfig.APIKeys field access
```

14:30-17:00 - Implement missing functions

```
cd HelixCode
```

```
# Create config/api_key_manager.go if needed
```

```
# Define all Strategy constants
```

```
# Add APIKeys field to config struct
```

Deliverables Day 2:

☐

memory_mock.go compilation fixed

☐

API key integration test compilation fixed

☐

All mock files building successfully

☐

Test infrastructure restored

DAY 3: ISOLATED FILES CLEANUP

Date: Day 3 of implementation

Objective: Fix isolated_files package conflicts and missing dependencies

Morning Session (4 hours)

```
# Task 3.1: Fix package import issues
```

```
cd /media/milosvasic/DATA4TB/Projects/HelixCode
```

09:00-10:30 - Analyze isolated_files conflicts

```
ls -la isolated_files/
```

```
# Found: packages integration and cognee conflict
```

10:30-12:00 - Restructure isolated files

```
# Move integration tests to tests/integration/
```

```
# Move cognee files to proper location
```

```
# Fix all import paths
```

Afternoon Session (4 hours)

```
# Task 3.2: Fix missing internal packages
```

```
cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
```

```
13:00-15:00 - Create missing internal packages
mkdir -p internal/hardware internal/cognee internal/provider
# Create stub implementations if needed
# Add go.mod entries
```

```
15:00-17:00 - Test all isolated files
go build ./isolated_files/...
go test -c ./isolated_files/...
```

Deliverables Day 3:

- ☐ isolated_files conflicts resolved
 - ☐ Missing packages created/linked
 - ☐ All isolated files compiling
 - ☐ Package structure cleaned up
-

DAY 4: BUILD SYSTEM VALIDATION

Date: Day 4 of implementation

Objective: Achieve clean build across entire project

Morning Session (4 hours)

```
# Task 4.1: Test complete build
cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
```

```
09:00-10:30 - Full project build test
go build -v ./... 2>&1 | tee build_validation.log
# Document any remaining errors
```

```
10:30-12:00 - Fix remaining compilation issues
# Address any new errors discovered
# Fix import cycles if any
# Ensure all dependencies resolved
```

Afternoon Session (4 hours)

```
# Task 4.2: Validate make targets
cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
```

13:00-15:00 - Test make commands

`make clean`

`make logo-assets`

`make build`

`make test`

15:00-17:00 - Document build process

Update README with build requirements

Create build troubleshooting guide

Verify Docker build works

Deliverables Day 4:

☐

Clean build achieved (go build ./...)

☐

All make targets working

☐

Build documentation updated

☐

Docker build validated

DAY 5: TEST INFRASTRUCTURE RESTORATION

Date: Day 5 of implementation

Objective: Enable and fix skipped tests

Morning Session (4 hours)

Task 5.1: Analyze skipped tests

`cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode`

09:00-10:30 - Find all skipped tests

`grep -r "t.Skip" . --include="*.go" | tee skipped_tests.log`

Analyze 32 skipped tests identified

10:30-12:00 - Categorize skip reasons

- Deprecated tests (remove)

- Broken tests (fix)

- Expensive tests (keep skipped with flag)

- Platform-specific tests (conditional)

Afternoon Session (4 hours)

Task 5.2: Fix critical skipped tests

```
cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
```

13:00-15:00 - Fix high-priority skipped tests

Focus on core functionality tests

Authentication tests

Memory provider tests

LLM integration tests

15:00-17:00 - Remove obsolete tests

Delete deprecated test files

Clean up test utilities

Update test documentation

Deliverables Day 5:

☐

Skipped tests analyzed and categorized

☐

Critical skipped tests fixed/enabled

☐

Obsolete tests removed

☐

Test execution rate >95%

DAY 6: BASIC TEST VALIDATION

Date: Day 6 of implementation

Objective: Validate core test suites are working

Morning Session (4 hours)

Task 6.1: Run unit test suite

```
cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
```

09:00-10:30 - Execute unit tests

```
go test -v ./internal/... -short | tee unit_test_results.log
```

Focus on core packages first

10:30-12:00 - Fix failing unit tests

Address any test failures

```
# Update test mocks if needed
# Ensure deterministic tests
```

Afternoon Session (4 hours)

```
# Task 6.2: Validate integration tests
cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode

13:00-15:00 - Run integration tests
go test -v ./tests/integration/... -short | tee
           integration_test_results.log
# Test with mock dependencies

15:00-17:00 - Security and performance tests
go test -v ./tests/security/... | tee security_test_results.log
go test -v ./tests/performance/... -short | tee
           performance_test_results.log
```

Deliverables Day 6:

- ☐ Unit tests passing (>95% success rate)
 - ☐ Integration tests validated
 - ☐ Security tests passing
 - ☐ Performance tests baseline established
-

DAY 7: PHASE 0 COMPLETION VALIDATION

Date: Day 7 of implementation

Objective: Validate Phase 0 completion and prepare for Phase 1

Morning Session (4 hours)

```
# Task 7.1: Complete validation
cd /media/milosvasic/DATA4TB/Projects/helix_code/HelixCode

09:00-10:30 - Final build test
go build -v ./...
make clean && make build
# Must achieve 100% clean build
```

```
10:30-12:00 - Test suite validation
./run_tests.sh --unit --integration --security
# Document any remaining issues
# Create Phase 0 completion report
```

Afternoon Session (4 hours)

```
# Task 7.2: Phase 1 preparation
cd /media/milosvasic/DATA4TB/Projects/HelixCode
```

```
13:00-15:00 - Create E2E test framework analysis
# Analyze existing E2E framework
# Identify gaps and requirements
# Create test case specifications
```

```
15:00-17:00 - Planning and documentation
# Update implementation tracker
# Create Phase 1 detailed plan
# Document lessons learned
# Prepare for next phase
```

Deliverables Day 7:

- ☐ Phase 0 completion validated
- ☐ Clean build achieved (100%)
- ☐ Core tests passing (>95%)
- ☐ Phase 1 plan created
- ☐ Phase 0 report generated

PHASE 0 SUCCESS METRICS

Daily Progress Tracking:

Day 1: Build error catalog complete, X11 dependencies fixed
Day 2: Mock compilation errors resolved
Day 3: Isolated files conflicts resolved
Day 4: Clean build achieved across project
Day 5: Test infrastructure restored (>95% execution)

Day 6: Core test suites validated and passing

Day 7: Phase 0 completion validated, Phase 1 planned

Quantitative Metrics:

Before Phase 0:

- Compilation errors: 15+
- Test execution rate: 65%
- Build success: 0%
- Skipped tests: 32

After Phase 0 (Target):

- Compilation errors: 0
- Test execution rate: >95%
- Build success: 100%
- Skipped tests: <5 (only expensive/platform-specific)

Quality Gates:

- Gate 1: No compilation errors (go build ./...)
 - Gate 2: All make targets working (make build, make test)
 - Gate 3: Test execution rate >95%
 - Gate 4: Core functionality tests passing
 - Gate 5: Documentation updated for build process
-

CRITICAL RISKS & MITIGATION

Risk 1: Complex Dependency Resolution

- **Mitigation:** Systematic approach, fix one component at a time
- **Contingency:** Create Docker environment with all dependencies

Risk 2: Mock Implementation Complexity

- **Mitigation:** Start with basic implementations, enhance incrementally
- **Contingency:** Use simpler mock patterns initially

Risk 3: Test Interdependencies

- **Mitigation:** Isolate tests, use proper mocking
- **Contingency:** Refactor tests for better isolation

Risk 4: Time Overrun

- **Mitigation:** Daily progress tracking, early issue escalation
- **Contingency:** Extend timeline by 2-3 days if needed



DAILY STANDUP TEMPLATE

What I completed yesterday:

☐

Task 1: _____

☐

Task 2: _____

☐

Deliverables: _____

What I’m working on today:

☐

Current task: _____

☐

Expected deliverables: _____

☐

Blockers: _____

Blockers/Impediments:

- Issue: _____
- Impact: _____
- Help needed: _____



NEXT PHASE PREVIEW

Phase 1: Test Completion (Weeks 2-4)

Objective: Achieve 100% test coverage across all 6 test types - **Week 2:** E2E test implementation (90 new tests) - **Week 3:** Integration and performance tests - **Week 4:** Coverage expansion and validation

Phase 2: Documentation (Weeks 5-6)

Objective: Complete all missing documentation - **9 critical documentation files** - **Enhanced user manual** - **API reference completion**

Phase 3: Video Courses (Weeks 7-9)

Objective: Create 50 professional videos (7.5 hours) - **Professional recording and editing** - **Website integration** - **Certificate system**

CURRENT STATUS: **IN PROGRESS** - Day 1 of Phase 0

NEXT MILESTONE: **Phase 0 Completion** - Clean build and test infrastructure

ESTIMATED COMPLETION: **7 days** (Week 1 of 11 total weeks)

Tracker updated: December 11, 2025 - Implementation begins